

Module 2: Setup & Initialization

2.1 Installing Git

Windows

1. Open your browser and go to git-scm.com
2. Click **Download** → Choose **Windows**
3. Select either **32-bit** or **64-bit** based on your system
4. Run the installer and follow the prompts (defaults are fine)

macOS

Option 1 — Homebrew (Recommended):

```
brew install git
```

Option 2 — Installer:

1. Go to git-scm.com → Download → macOS
2. Follow the installation instructions

Linux (Ubuntu/Debian)

```
sudo apt update  
sudo apt install git
```

Linux (Fedora/RHEL)

```
sudo dnf install git
```

2.2 Terminal Setup

After installing Git, you need a terminal to run commands.

OS	Terminal Options
Windows	Command Prompt, PowerShell, or Git Bash (recommended)
macOS	Terminal app (built-in)
Linux	Any terminal emulator

Tip: Many developers prefer **Git Bash** on Windows because it feels similar to a Linux terminal and makes running Git commands easier and more intuitive.

2.3 Verifying Installation

Open your terminal and type:

```
git --version
```

Expected output:

```
git version 2.45.1
```

The version number may differ depending on when you installed Git. If you see a version number, Git is installed correctly. If you get an error, Git is not installed — go back and install it.

2.4 Essential Terminal Commands (Non-Git)

These are **general terminal commands** — not Git commands — but you'll use them all the time:

Command	Description	Example
<code>cd <folder></code>	Change directory (move into a folder)	<code>cd Desktop</code>
<code>cd ..</code>	Move one level up	<code>cd ..</code>
<code>pwd</code>	Print working directory (show current path)	<code>pwd</code>
<code>ls</code>	List files and folders	<code>ls</code>
<code>ls -la</code>	List all files (including hidden ones)	<code>ls -la</code>
<code>mkdir <name></code>	Create a new directory	<code>mkdir my-project</code>
<code>touch <name></code>	Create a new empty file	<code>touch index.html</code>
<code>rm <file></code>	Delete a file	<code>rm old.txt</code>
<code>rm -r <folder></code>	Delete a folder recursively	<code>rm -r my-folder</code>

Windows Note: Some commands differ on Windows CMD. For example, use `dir` instead of `ls`, and `type nul >` instead of `touch`. **Git Bash** on Windows supports all the Linux-style commands above.

2.5 Creating a Local Project

Step-by-Step Initial Setup Example:

Step	Command	Directory State	File Status (Git Context)	Description & Internal Action
1	<code>cd Desktop</code>	<code>/Desktop</code>	Untracked	Moves the terminal focus to the user's Desktop directory.
2	<code>mkdir git-1</code>	<code>/Desktop/git-1</code>	Untracked	Creates a new normal OS directory named <code>git-1</code> .
3	<code>cd git-1</code>	<code>/Desktop/git-1</code>	Untracked	Enters the new project folder.
4	<code>touch 1.txt</code> <code>2.txt</code>	<code>/Desktop/git-1</code>	Untracked	Creates two empty text files <code>1.txt</code> and <code>2.txt</code> in the root.
5	<code>mkdir my-</code> <code>folder</code> <code>touch my-</code> <code>folder/3.txt</code>	<code>/Desktop/git-1/my-folder</code>	Untracked	Creates a subfolder <code>my-folder</code> containing <code>3.txt</code> .

Result — your local folder structure:

```
git-1/
├── 1.txt
├── 2.txt
└── my-folder/
    └── 3.txt
```

2.6 `git init` — Initializing a Repository

Once you have a project folder, you need to tell Git: **"Start tracking this folder."**

Command:

```
git init
```

Command Output:

```
Initialized empty Git repository in /Users/you/Desktop/git-1/.git/
```

Detailed Step-by-Step Status & Internals:

Action / Check	Command	System Status	Git Internal Functionality
Initialize Repo	<code>git init</code>	Directory is now a local Git repository	Creates the hidden <code>.git/</code> folder containing standard metadata config files.
Check Hidden Files	<code>ls -la</code>	Displays files plus the hidden <code>.git</code> folder	Shows that Git has successfully mounted itself in this directory.
Check Project Status	<code>git status</code>	Lists all files as Untracked	Git scans the workspace and detects <code>1.txt</code> , <code>2.txt</code> , and <code>my-folder/</code> but isn't watching changes yet.

Inside the hidden `.git` folder:

When you run `git init`, Git initializes several key files and directories:

- `objects/` — The database where Git stores all compressed code blobs, trees, and commit metadata.
- `refs/` — Stores pointers to branches and tags (e.g., `refs/heads/main`).
- `config` — Project-specific Git configuration settings (remotes, branch settings, local configurations).
- `HEAD` — A file pointing to the current active branch (e.g. `ref: refs/heads/main` or `master`).
- `description` — Used only by the GitWeb program to describe the repo.

⚠ **Never manually edit or delete the `.git` folder** — it would destroy your entire version history.

From this point on, Git will track **every change** you make inside this folder.

2.7 Configuring Git: User Name & Email

Why is this needed?

When you make your **first commit**, Git needs to know **who** is making the changes. This information is attached to every commit in the project history.

If you haven't configured it, you'll see:

```
Please tell me who you are.
```

Setting up globally (applies to all projects on your machine):

```
git config --global user.email "your-email@example.com"
git config --global user.name "Your Name"
```

Setting up locally (applies only to the current project):

```
git config --local user.email "your-email@example.com"
git config --local user.name "Your Name"
```

Verify your configuration:

```
git config --global user.name
git config --global user.email
```

--global vs --local

Flag	Scope	Use when...
<code>--global</code>	Entire computer (all repos)	You use the same identity everywhere
<code>--local</code>	Current repository only	You use a different email for work vs personal

Tip: Set `--global` once when you install Git. Use `--local` only when you need a different identity for a specific project (e.g., work email vs personal email).

2.8 `git clone` — Cloning a Remote Repository

Instead of initializing a new repository from scratch, you can **clone** (download) an existing one from GitHub.

Step-by-step:

1. Go to the repository on GitHub
2. Click the green **Code** button
3. Copy the **HTTPS** link
4. In your terminal:

```
git clone https://github.com/username/repository-name.git
```

Output:

```
Cloning into 'repository-name' ...
remote: Enumerating objects: 10, done.
remote: Counting objects: 100% (10/10), done.
Receiving objects: 100% (10/10), done.
```

What happens when you clone?

- A new folder is created with the repository name
- All files from the remote repo are downloaded
- A `.git` folder is created (the repo is fully initialized)

- The remote connection (called `origin`) is automatically set up

```
# Enter the cloned folder
cd repository-name

# See the files
ls

# Verify it's a Git repo (hidden .git folder exists)
ls -a
```

Two ways to create a Git repository:

Method	Command	When to use
Initialize locally	<code>git init</code>	Starting a brand-new project
Clone from remote	<code>git clone <url></code>	Joining an existing project

No matter which method you choose, **initialization is always required** before you can start working with Git.

2.9 Creating a Remote Repository on GitHub

Step-by-step:

1. Go to github.com and log in (create an account if needed)
2. Click the **green "New"** button (or the `+` icon → "New repository")
3. Fill in:
 - **Repository name** — e.g., `git-journey`
 - **Description** — e.g., "We are learning Git and GitHub"
 - **Visibility** — Public or Private
4. Click **Create repository**

The repository is now created but **empty**. You can add files via the GitHub web interface or push files from your local machine.

Key Takeaways

1. Install Git from git-scm.com and verify with `git --version`
 2. Use `git init` to start tracking a new project folder
 3. Use `git clone <url>` to download an existing repository
 4. Configure your name and email with `git config --global`
 5. The hidden `.git` folder stores all tracking data — never delete it
 6. You can create repositories **locally** (init) or on **GitHub** (clone)
-